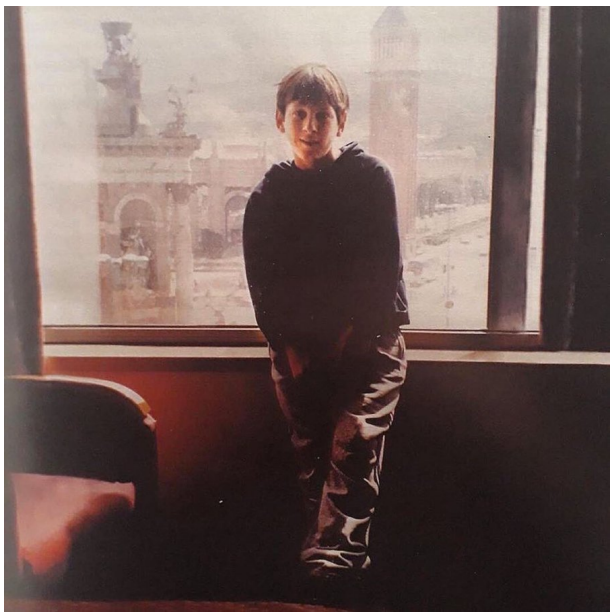

Protocol Audit Report

Version 1.0

maheshbsl

October 23, 2024

Protocol Audit Report

maheshbsl

March 7, 2023

Prepared by: maheshbsl Lead Security Researcher: - maheshbsl

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    * [H-1] `PasswordStore::s_password` is stored onchain, which is visible to everyone.
  - Likelihood of Impact:
    * [H-2] `PasswordStore::setPassword` has no access control mechanism applied, that would lead to anyone(non-owner) can change the password of others
  - Likelihood of Impact:

Protocol Summary

PasswordStore is a protocol to store passwords onchian. Protocol is designed to be used by single user, only the owner should be able to set and retrieve passwords.

Disclaimer

The mahshbsl team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings decribed in the document correspond to the following commit hash:

```
2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Scope

```
./src/
## PasswordStore.sol
```

Roles

-Owner -> Should be able to store and retrieve password. -Outsiders -> Noone should be able to store and retrieve password.

Executive Summary

Issues found

Serverity	Number of Issues Found
High	2
Medium	0
Low	0
Informational	0
Gas	0
Total	2

Findings

High

[H-1] `PasswordStore::s_password` is stored onchain, which is visible to everyone.

Description: The contract stores a sensitive password in the state variable `s_password` marked as private. However, on the Ethereum blockchain, all data, including private variables, can be publicly accessed via blockchain explorers or by directly querying storage slots. This exposes the password to anyone with the technical know-how to retrieve it, resulting in a significant security risk.

Impact: Anyone can read the password.Breaking the protocol.

Proof of Concept:

This below test case shows how anyone can read password directly from blockchain.

1. Create a locally running chain

```
make anvil
```

2. Deploy the contract on the chain

```
make deploy
```

3. Run the storage tool

We use 1 because that's the storage slot of the `s_password` in the contract.

```
cast storage <contract address>  1 --rpc-url http://localhost:8545
```

4. You should receive an output look like this `0x6d7950617373776f72640000000000000000000000000000`

5. bytes to string

```
cast parse-bytes32-string 0
    x6d7950617373776f72640000000000000000000000000000000000000000000014
```

6. Output `myPassword`

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the stored password. However, you're also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with this decryption key.

Likelihood of Impact:

– Impact: HIGH – Likelihood: HIGH – Severity: HIGH

[H-2] `PasswordStore::setPassword` has no access control mechanism applied, that would lead to anyone(non-owner) can change the password of others

Description:

Impact:

Proof of Concept: Add this in the PasswordStore.t.sol.

Code

```
function test_anyone_can_set_password(address randomAddress) public {
        vm.assume(randomAddress != owner);
        vm.prank(randomAddress);
        string memory expectedPassword = "myNewPassword";
        passwordStore.setPassword(expectedPassword);
        vm.prank(owner);
        string memory actualPassword = passwordStore.getPassword();
        assertEq(expectedPassword, actualPassword);
    }
```

Recommended Mitigation: Add an access control mechanism to the `PasswordStore::setPassword` function.

```
if (msg.sender != s_owner) {
    revert PasswordStore_NotOwner();
}
```

Likelihood of Impact:

– Impact: HIGH – Likelihood: HIGH – Severity: HIGH